10 Monoids

By the end of part 2, we were getting comfortable with considering data types in terms of their *algebras*—that is, the operations they support and the laws that govern those operations. Hopefully you will have noticed that the algebras of very different data types tend to share certain patterns in common. In this chapter, we'll begin identifying these patterns and taking advantage of them.

This chapter will be our first introduction to *purely algebraic* structures. We'll consider a simple structure, the *monoid*,[1] which is defined *only by its algebra*. Other than satisfying the same laws, instances of the monoid interface may have little or nothing to do with one another. Nonetheless, we'll see how this algebraic structure is often all we need to write useful, polymorphic functions.

We choose to start with monoids because they're simple, ubiquitous, and useful. Monoids come up all the time in everyday programming, whether we're aware of them or not. Working with lists, concatenating strings, or accumulating the results of a loop can often be phrased in terms of monoids. We'll see how monoids are useful in two ways: they facilitate parallel computation by giving us the freedom to break our problem into chunks that can be computed in parallel; and they can be composed to assemble complex calculations from simpler pieces.

10.1 What is a monoid?

Let's consider the algebra of string concatenation. We can add `"foo" + "bar"` to get `"foobar"`, and the empty string is an *identity element* for that operation. That is, if we say `(s + "")` or `("" + s)`, the result is always `s`. Furthermore, if we combine three

[1] The name *monoid* comes from mathematics. In category theory, it means a category with one object. This mathematical connection isn't important for our purposes in this chapter, but see the chapter notes for more information.

strings by saying `(r + s + t)`, the operation is *associative*—it doesn't matter whether we parenthesize it: `((r + s) + t)` or `(r + (s + t))`.

The exact same rules govern integer addition. It's associative, since `(x + y) + z` is always equal to `x + (y + z)`, and it has an identity element, `0`, which "does nothing" when added to another integer. Ditto for multiplication, whose identity element is `1`.

The Boolean operators `&&` and `||` are likewise associative, and they have identity elements `true` and `false`, respectively.

These are just a few simple examples, but algebras like this are virtually everywhere. The term for this kind of algebra is *monoid*. The laws of associativity and identity are collectively called the *monoid laws*. A monoid consists of the following:

- Some type `A`
- An associative binary operation, `op`, that takes two values of type `A` and combines them into one: `op(op(x,y), z) == op(x, op(y,z))` for any choice of `x: A`, `y: A`, `z: A`
- A value, `zero: A`, that is an identity for that operation: `op(x, zero) == x` and `op(zero, x) == x` for any `x: A`

We can express this with a Scala trait:

```
trait Monoid[A] {
  def op(a1: A, a2: A): A
  def zero: A
}
```

Satisfies `op(op(x,y), z) == op(x, op(y,z))`

Satisfies `op(x, zero) == x` and `op(zero, x) == x`

An example instance of this trait is the `String` monoid:

```
val stringMonoid = new Monoid[String] {
  def op(a1: String, a2: String) = a1 + a2
  val zero = ""
}
```

List concatenation also forms a monoid:

```
def listMonoid[A] = new Monoid[List[A]] {
  def op(a1: List[A], a2: List[A]) = a1 ++ a2
  val zero = Nil
}
```

The purely abstract nature of an algebraic structure

Notice that other than satisfying the monoid laws, the various `Monoid` instances don't have much to do with each other. The answer to the question "What is a monoid?" is simply that a monoid is a type, together with the monoid operations and a set of laws. A monoid is the algebra, and nothing more. Of course, you may build some other intuition by considering the various concrete instances, but this intuition is necessarily imprecise and nothing guarantees that all monoids you encounter will match your intuition!

EXERCISE 10.1

Give `Monoid` instances for integer addition and multiplication as well as the Boolean operators.

```
val intAddition: Monoid[Int]
val intMultiplication: Monoid[Int]
val booleanOr: Monoid[Boolean]
val booleanAnd: Monoid[Boolean]
```

EXERCISE 10.2

Give a `Monoid` instance for combining `Option` values.

```
def optionMonoid[A]: Monoid[Option[A]]
```

EXERCISE 10.3

A function having the same argument and return type is sometimes called an *endofunction*.[2] Write a monoid for endofunctions.

```
def endoMonoid[A]: Monoid[A => A]
```

EXERCISE 10.4

Use the property-based testing framework we developed in part 2 to implement a property for the monoid laws. Use your property to test the monoids we've written.

```
def monoidLaws[A](m: Monoid[A], gen: Gen[A]): Prop
```

Having versus being a monoid

There is a slight terminology mismatch between programmers and mathematicians when they talk about a type *being* a monoid versus *having* a monoid instance. As a programmer, it's tempting to think of the instance of type `Monoid[A]` as *being* a monoid. But that's not accurate terminology. The monoid is actually both things—the type together with the instance satisfying the laws. It's more accurate to say that the

[2] The Greek prefix *endo-* means *within*, in the sense that an endofunction's codomain is within its domain.